Deep Reinforcement Learning

Multi-armed bandits

Prof. Dr. Sebastian Peitz

Chair of Safe Autonomous Systems, TU Dortmund

Summer term 2026

Content

Content

- Multi-armed bandits
- Action-value methods
- A simple bandit algorithm
- What we have not discussed
- Some next steps

Where are we?

Chapter	Topic	Content
	Basics & tabular methods	
1	Multi-armed bandits	Exploration-exploitation dilemma
2	Markov decision processes	
3	Dynamic programming	
4	Monte Carlo methods	
5	Temporal difference learning & Q-learning	
	Deep-learning-based methods	
	Model-Based Control	
	Advanced Topics	

Lecture contents

Example: Route planning (OH16 to Hansaplatz by car)

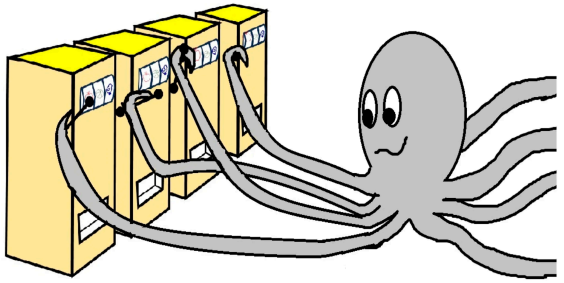
Multi-armed bandits

Multi-armed bandits

- Let us assume that we have a slot machine and we repeatedly can choose between k different actions
- After each choice a_t you receive a numerical reward r_t chosen from a stationary probability distribution*
- Objective: maximize the **expected total reward** over some time period (e.g., over 1000 action selections, or *time steps*)
- We refer to this as the **value**:

$$q(a) = \mathbb{E}\left[r_t | a_t = a\right]$$

- If we knew $q(a)$, then it would be easy to choose!
- Instead, we have to rely on estimates $Q_t(a)$ which we can iteratively update based on past experience



A multi-armed bandit (Ferreira, Schubert, and Scotti 2024)

*We will have a discussion on the notation and random variables in the next lecture (MDPs)

Action-value methods

Action-value methods

- *Action-value methods*: estimate the values of actions and then use these estimates to make action selection decisions.
- What was the true value again? $\rightarrow$ the **expected reward**
- What does that mean for a finite number of samples? $\rightarrow$ we take the **mean**

$$Q_t(a) = \frac{\sum_{i=1}^{t-1} r_i \cdot \mathbb{1}_{a_i=a}}{\sum_{i=1}^{t-1} \mathbb{1}_{a_i=a}}.$$

  Here, $\mathbb{1}_x$ denotes the random variable that is 1 if the predicate x is true and 0 if x is false. (In other words, we're *counting* how often the action a was taken).
- For $\sum_{i=1}^{t-1} \mathbb{1}_{a_i=a} = 0$ (we have never used the action a), we can choose a default value such as $Q_t(a) = 0$.
- We call this the *sample-average method* (Sutton and Barto 1998).

Question: How do we pick the action a_t? $\Rightarrow$ just take the max!

$$a_t = \arg\max_a Q_t(a),$$

(with ties broken arbitrarily).

A simple bandit algorithm

The k-armed testbed

- Let's study a large number of randomly generated k-armed bandit problems (here: $k = 10$).
- For each bandit problem the action values $q(a)$, $a = 1, \dots, 10$, are selected to a normal distribution:

$$q(a) \sim \mathcal{N}(0, 1).$$

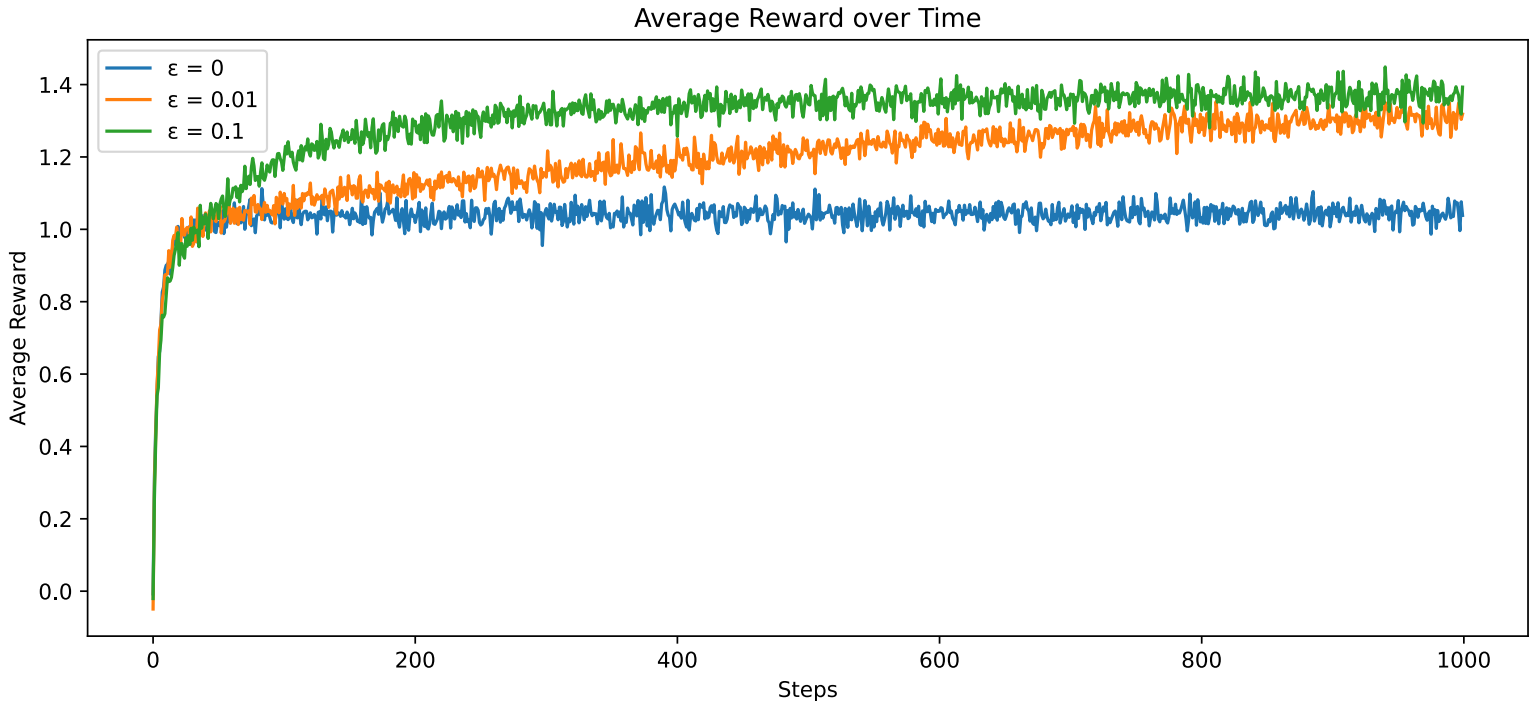
- The corresponding reward is a random variable as well:

$$r_t \sim \mathcal{N}(q(a_t), 1).$$

- One **run**: perform $T = 1000$ steps. For each time step, report the average reward received until then.
- **Statistics**: repeat the above experiment 2000 times.

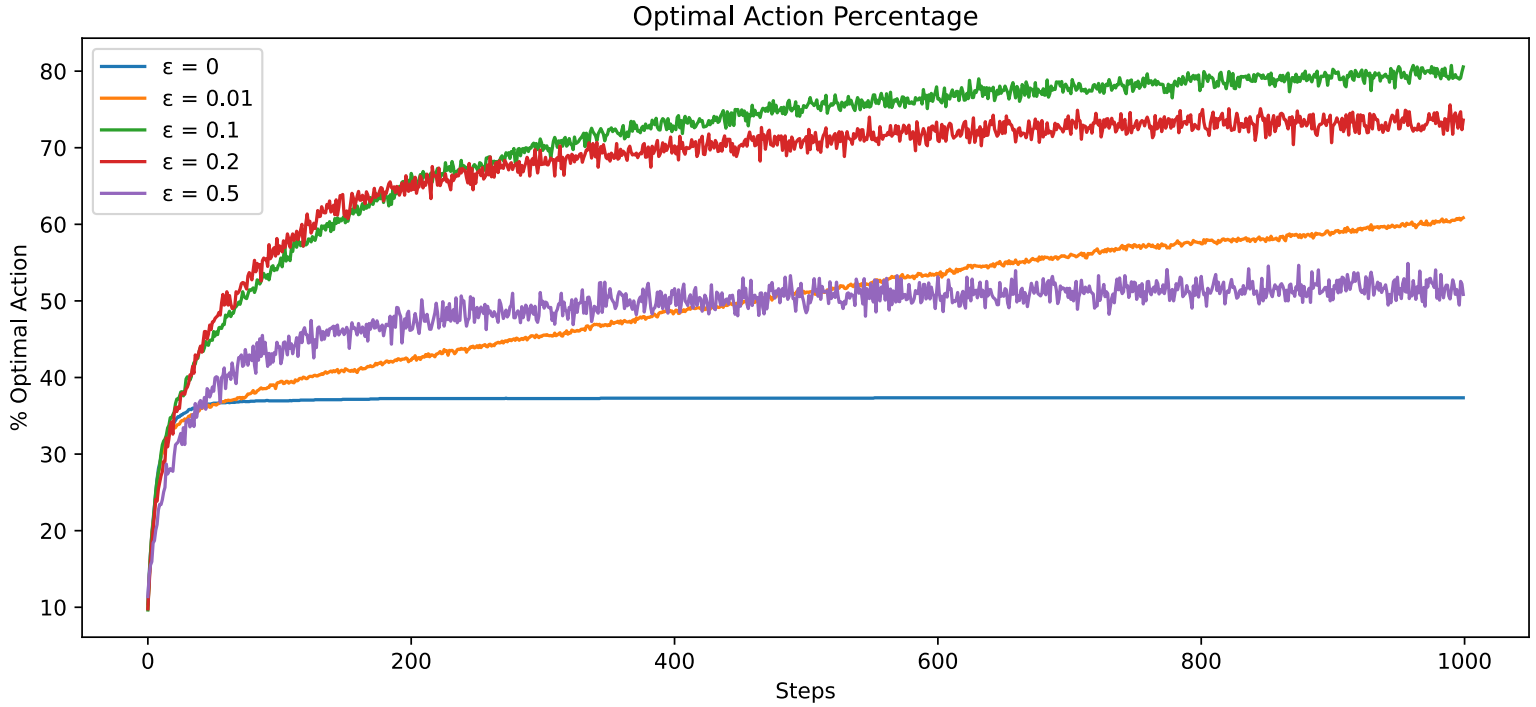
The k-armed testbed: rewards

- First, we are going to follow a **greedy** strategy: pick the maximizing action *always*.
- Then, we will compare against ϵ greedy:
  - pick the maximizing action $1 - \epsilon$ of the time,
  - pick a random action ϵ of the time.



What is ϵ's job? $\Rightarrow$ the **exploration**

The k-armed testbed: optimal actions



- Too little (or no) exploration is harmful.
- Too much exploration is also harmful.
- Should we think about a **schedule** in terms of the exploration?

A simple bandit algorithm

```
# Initialization
for i in range(k):
  Q(a) = 0
  N(a) = 0

# Run forever
while(True):

  # exploration versus exploitation
  if rand() > epsilon:
    a = argmax(Q)        # exploitation
  else:
    a = randint(k)       # exploration

  r = bandit(a)

  # Error correction towards target r
  N(a) = N(a) + 1
  Q(a) = Q(a) + (1/N(a)) * (r - Q(a))
```

A simple version of the ϵ greedy bandit

Some next steps

Computing $Q(a)$ on the fly

Calculating $Q(a)$ anew every time appears to be expensive. For a single action, consider that we have received a sequence of rewards $r_1, \dots, r_{t-1}$ so far:

$$Q_t = \frac{r_1 + r_2 + \dots + r_{t-1}}{t-1}.$$

Wouldn't it be easier if we could just updated incrementally? $\Rightarrow$ let's try

$$\begin{aligned} Q_{t+1} &= \frac{1}{t}\sum_{i=1}^{t} r_i = \frac{1}{t}\left(r_t + \sum_{i=1}^{t-1} r_i\right) = \frac{1}{t}\left(r_t + (t-1)\frac{1}{t-1}\sum_{i=1}^{t-1} r_i\right) = \frac{1}{t}(r_t + (t-1)Q_t) = \frac{1}{t}(r_t + tQ_t - Q_t) \\ &= Q_t + \frac{1}{t}(r_t - Q_t)\,. \end{aligned}$$

The expression $[\textbf{target} - \textbf{OldEstimate}]$ is an **error estimate**, used to steer us closer to the **target**.
We will see a formulae of the type

$$\textbf{NewEstimate} \leftarrow \textbf{OldEstimate} + \mathit{StepSize}\,[\textbf{target} - \textbf{OldEstimate}]$$

frequently from now on!

Non-stationary problems

- In reinforcement learning, problems are often *non-stationary*, meaning that processes change over time (e.g., due to updates of the **policy** π).
- Recent rewards should be more relevant than those longer in the past!
- Let's define some constant **step size** $\alpha \in (0, 1]$:

$$\begin{aligned} Q_{t+1} = Q_t + \alpha[r_t - Q_t] &= \alpha r_t + (1-\alpha)Q_t \\ &= \alpha r_t + (1-\alpha)[\alpha r_{t-1} + (1-\alpha)Q_{t-1}] = \alpha r_t + (1-\alpha)\alpha r_{t-1} + (1-\alpha)^2 Q_{t-1} \\ &= \alpha r_t + (1-\alpha)\alpha r_{t-1} + (1-\alpha)^2 \alpha r_{t-2} + \ldots + (1-\alpha)^t r_1 \\ &= (1-\alpha)^t r_1 + \sum_{i=1}^{t} \alpha(1-\alpha)^{t-i} r_i. \end{aligned}$$

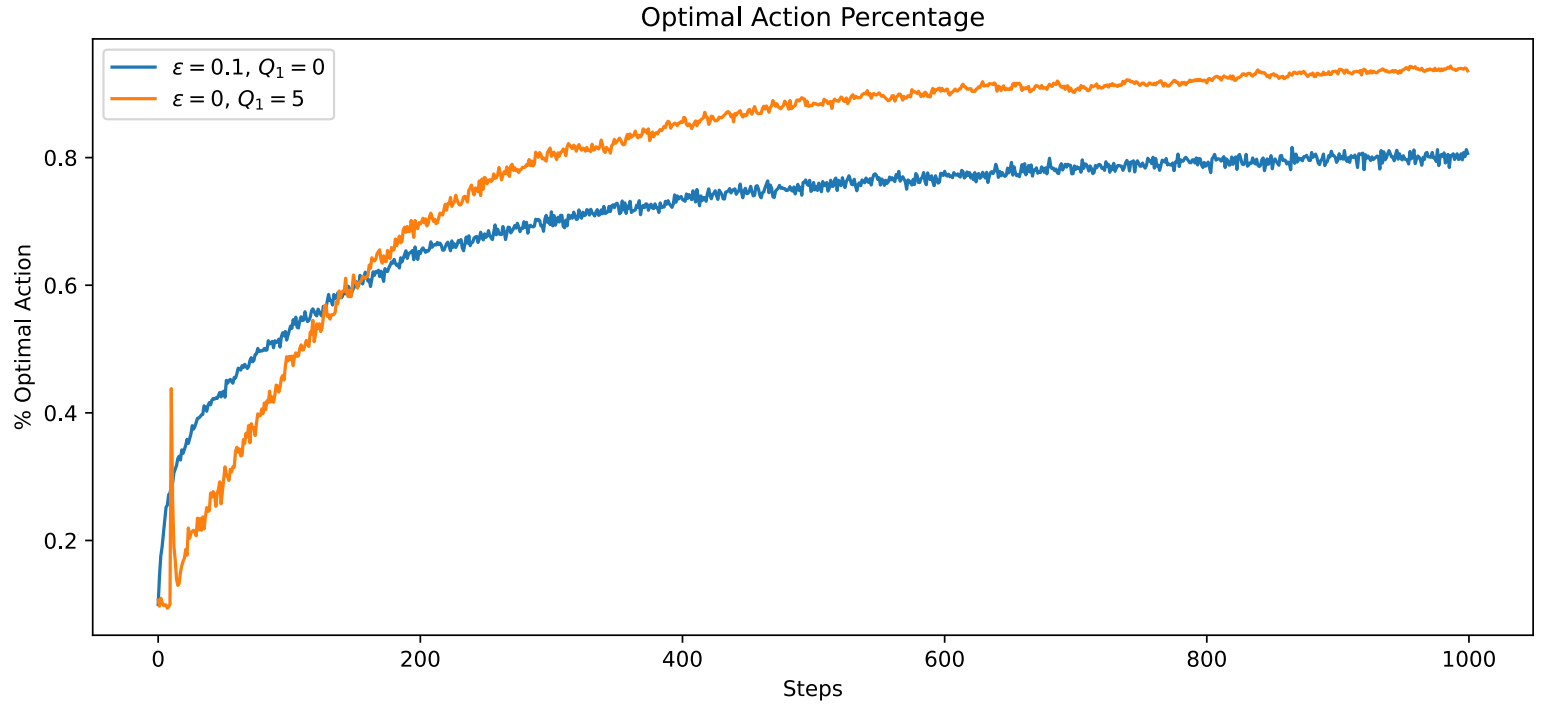
- This is called an *(exponential-recency) weighted average*, as

$$(1-\alpha)^t + \sum_{i=1}^{t} \alpha(1-\alpha)^{t-i} = 1.$$

- For time dependent weights α_t, we require $\sum_{i=1}^{\infty} \alpha_t(a) = \infty$ and $\sum_{i=1}^{\infty} \alpha_t^2(a) < \infty$ to ensure convergence. Does this hold for $\alpha_t = \frac{1}{t}$ and $\alpha_t = \alpha$? (Does it have to hold?)

Bias in the selection of initial values

- All methods up to now are *biased* in the sense that they depend on the (arbitrarily selected) initial guesses $Q_1(a)$.
- Can we use this to our advantage and increase exploration?
- Let us choose overly **optimistic initial values**!
- For $q(a) \sim \mathcal{N}(1, 0)$, an initial guess of $Q_1(a) = 5$ is certainly unrealistically high.



What we have not discussed

Two topics (and also many more) topics that we have not discussed:

- Bandits with **upper confidence bounds** (**UCB**):
  - Along with the estimate for Q_t, we assign an upper confidence bound based on the number of times N_t we have selected this action.
  - The larger N_t, the more confident we are in our estimate.
  - We do not select our next action according to the maximal estimate of Q_t, but to Q_t **plus** the confidence bound!
  - The less certain we are, the higher this bound will be $\Rightarrow$ improved exploration.
- Gradient bandits:
  - We assign a **preference** for each bandit arm a and introduce the probability $\pi_t(a)$ for choosing this arm.
  - We can then derive a gradient ascent scheme for the preferences.

For details, see (Sutton and Barto 1998)

Summary / what we have learned

- Multi-armed bandits model decision making problems where we want to maximize the expected return (the value) of an action in an unknown and uncertain environment.
- Action-value methods estimate the values of actions and then use these estimates to make action selection decisions.
- The exploration-exploitation dilemma describes the challenge of making use of what we know, versus needing to explore to further improve in the long run.
- A simple approach is the ϵ-greedy algorithm, where we explore randomly with proabability ϵ, and otherwise follow the optimal (greedy) action.

References

Ferreira, Rafael Pereira, Emil Schubert, and Américo Scotti. 2024. “Exploring Multi-Armed Bandit (MAB) as an AI Tool for Optimising GMA-WAAM Path Planning.” *Journal of Manufacturing and Materials Processing* 8 (3).

Sutton, R. S., and A. G. Barto. 1998. *Reinforcement Learning: An Introduction*. MIT Press.